

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB BACK END

ATIVIDADE PRÁTICA
API REST – BAOZI STORE

GABRIEL RODRIGUES CAVALIERI VALLE COSTA – RU: 5484519

RIO DE JANEIRO – RJ
2026

ATIVIDADE PRÁTICA

1. Descrição de uma situação fictícia

A Baozi Store é uma pequena loja que vende pão chinês (baozi). Para melhorar a organização do negócio, foi criado um sistema simples, em forma de API REST, para controlar clientes, produtos e pedidos.

Um cliente chamado **Gabriel5484519** realizou seu cadastro no sistema em 12/10/2026, data registrada no campo `clienteDesde`.

O produto vendido pela loja chama-se **Baozi de Porco** e é vendido por unidade, ao preço de R$ 8,50, estando disponível em estoque.

Em um determinado momento, o cliente Gabriel5484519 realizou um pedido de **6 unidades** do produto Baozi de Porco, totalizando R$ 51,00.

O sistema registra o cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja. Essas mesmas informações aparecem nos prints do Postman apresentados no item 4.

2. Diagrama caso de uso

O diagrama abaixo, em notação UML, mostra o ator principal (Usuário da API) e todos os casos de uso implementados: o CRUD completo de Cliente, Produto e Pedido. Ao registrar ou atualizar um pedido, a API inclui (<<include>>) a validação de que o cliente e o produto informados existem.

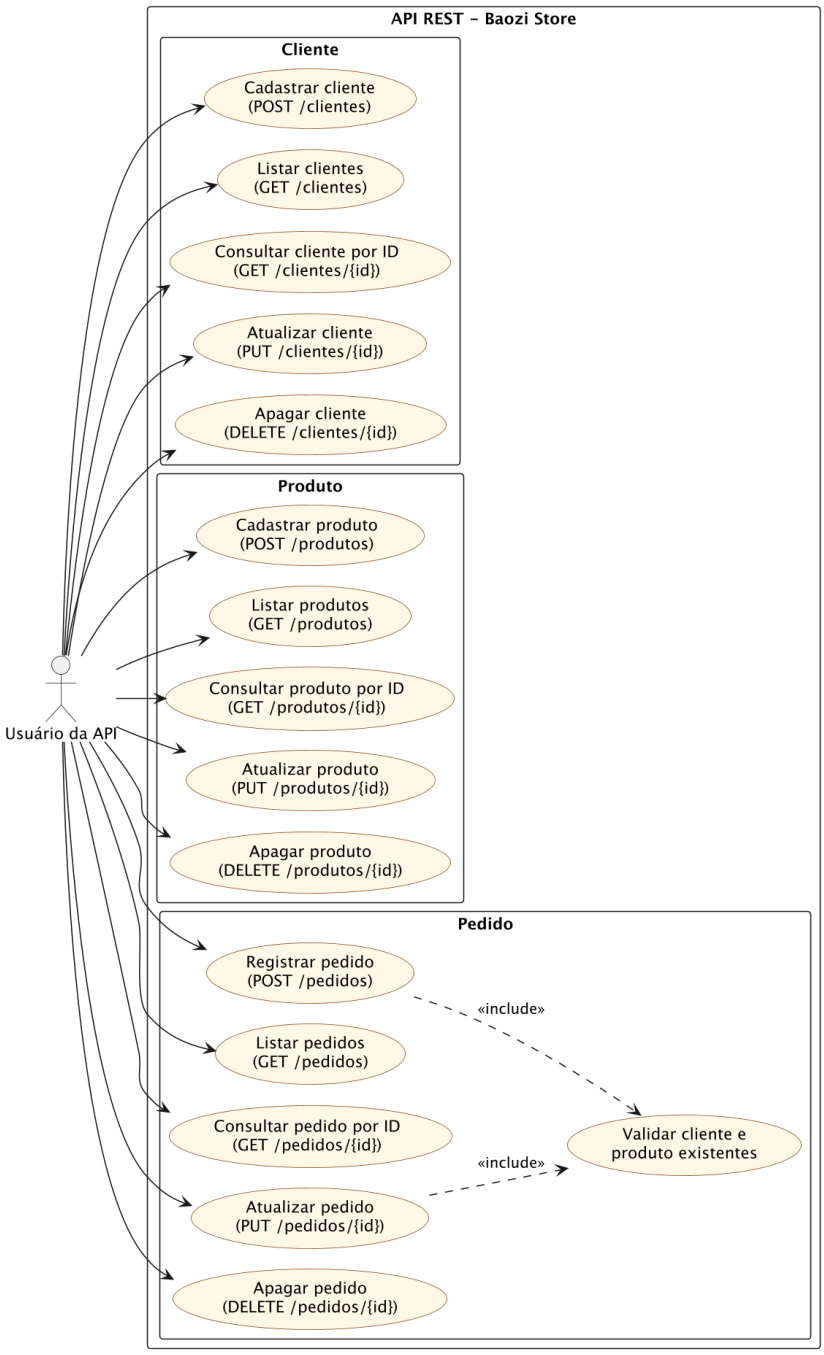


Figura 1 – Diagrama de caso de uso da API Baozi Store

3. Especificação da API desenvolvida

Tecnologias: Java 21, Spring Boot 3.5, Spring Data JPA, banco relacional H2 (padrão, salvo em arquivo) com perfil alternativo para MySQL, JSON em todos os endpoints e padrão MVC do Spring. O projeto está organizado nos pacotes `model`, `repository`, `controller` e `dto`.

3.1 Entidades e campos

Entidade	Campo	Tipo	Descrição
Cliente	id	Long	Identificador (gerado automaticamente)
Cliente	nome	String	Nome do cliente (Nome+RU)
Cliente	clienteDesde	LocalDate	Data de cadastro do cliente
Produto	id	Long	Identificador (gerado automaticamente)
Produto	nome	String	Nome do produto
Produto	preco	BigDecimal	Preço por unidade
Produto	estoque	Boolean	Indica se há o produto em estoque
Pedido	id	Long	Identificador (gerado automaticamente)
Pedido	clienteId	Long (FK)	Cliente que comprou (@ManyToOne, coluna cliente_id)
Pedido	produtoId	Long (FK)	Produto comprado (@ManyToOne, coluna produto_id)
Pedido	quantidade	Integer	Quantidade comprada (mínimo 1)

Relacionamentos: um Cliente pode ter vários Pedidos e um Produto pode estar em vários Pedidos (cada Pedido referencia exatamente um Cliente e um Produto).

3.2 Endpoints implementados

Método	Rota	Descrição	Status de resposta
POST	/clientes	Cadastra um cliente	201 Created
GET	/clientes	Lista todos os clientes	200 OK
GET	/clientes/{id}	Consulta cliente por ID	200 OK / 404
PUT	/clientes/{id}	Atualiza cliente (opcional)	200 OK / 404
DELETE	/clientes/{id}	Apaga cliente	204 / 404 / 409
POST	/produtos	Cadastra um produto	201 Created
GET	/produtos	Lista todos os produtos	200 OK
GET	/produtos/{id}	Consulta produto por ID	200 OK / 404
PUT	/produtos/{id}	Atualiza produto (opcional)	200 OK / 404
DELETE	/produtos/{id}	Apaga produto	204 / 404 / 409
POST	/pedidos	Registra um pedido	201 Created / 400
GET	/pedidos	Lista todos os pedidos	200 OK
GET	/pedidos/{id}	Consulta pedido por ID	200 OK / 404
PUT	/pedidos/{id}	Atualiza pedido (opcional)	200 OK / 400 / 404
DELETE	/pedidos/{id}	Apaga pedido	204 / 404

O status 409 (Conflict) é devolvido ao tentar apagar um cliente ou produto que possui pedidos vinculados; o 400 (Bad Request) indica dados inválidos ou cliente/produto inexistente no pedido.

3.3 Exemplos de JSON

POST /clientes

```
{ "nome": "Gabriel5484519", "clienteDesde": "2026-10-12" }
```

POST /produtos

```
{ "nome": "Baozi de Porco", "preco": 8.50, "estoque": true }
```

POST /pedidos

```
{ "clienteId": 1, "produtoId": 1, "quantidade": 6 }
```

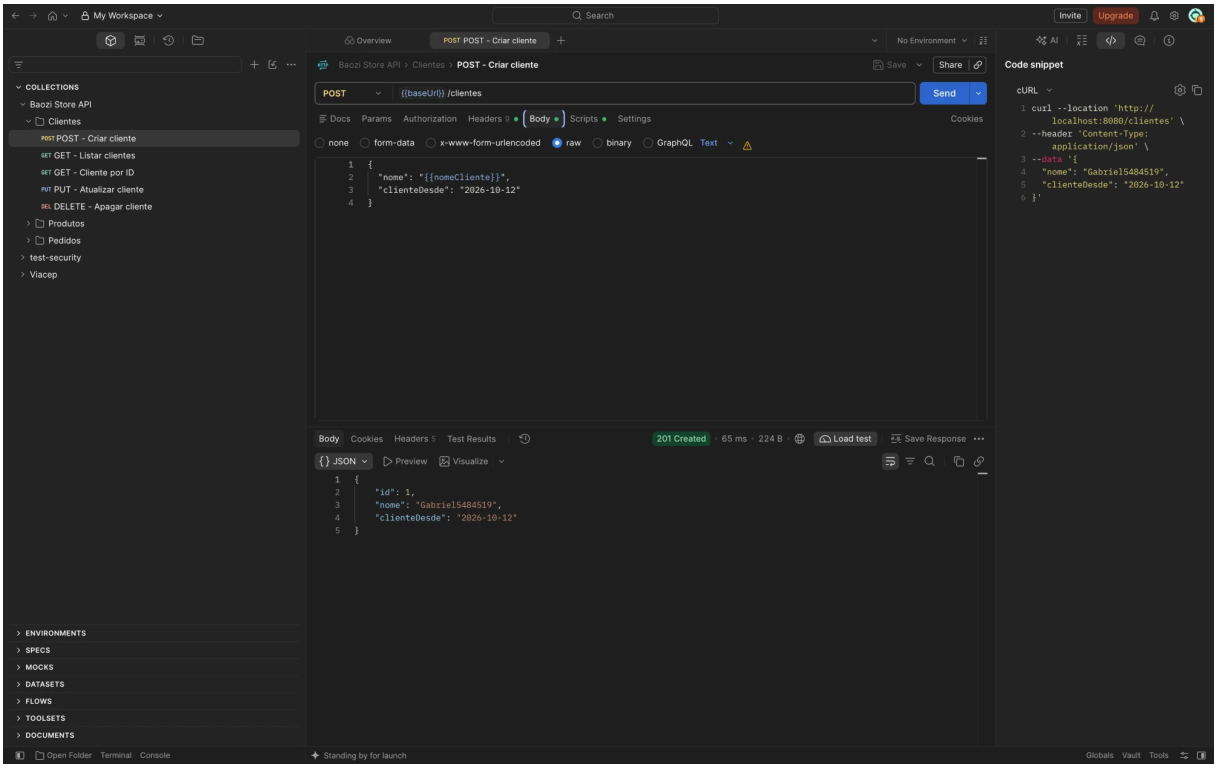
Resposta de GET /pedidos/1

```
{
  "id": 1,
  "cliente": { "id": 1, "nome": "Gabriel5484519", "clienteDesde": "2026-10-12" },
  "produto": { "id": 1, "nome": "Baozi de Porco", "preco": 8.50, "estoque": true },
  "quantidade": 6
}
```

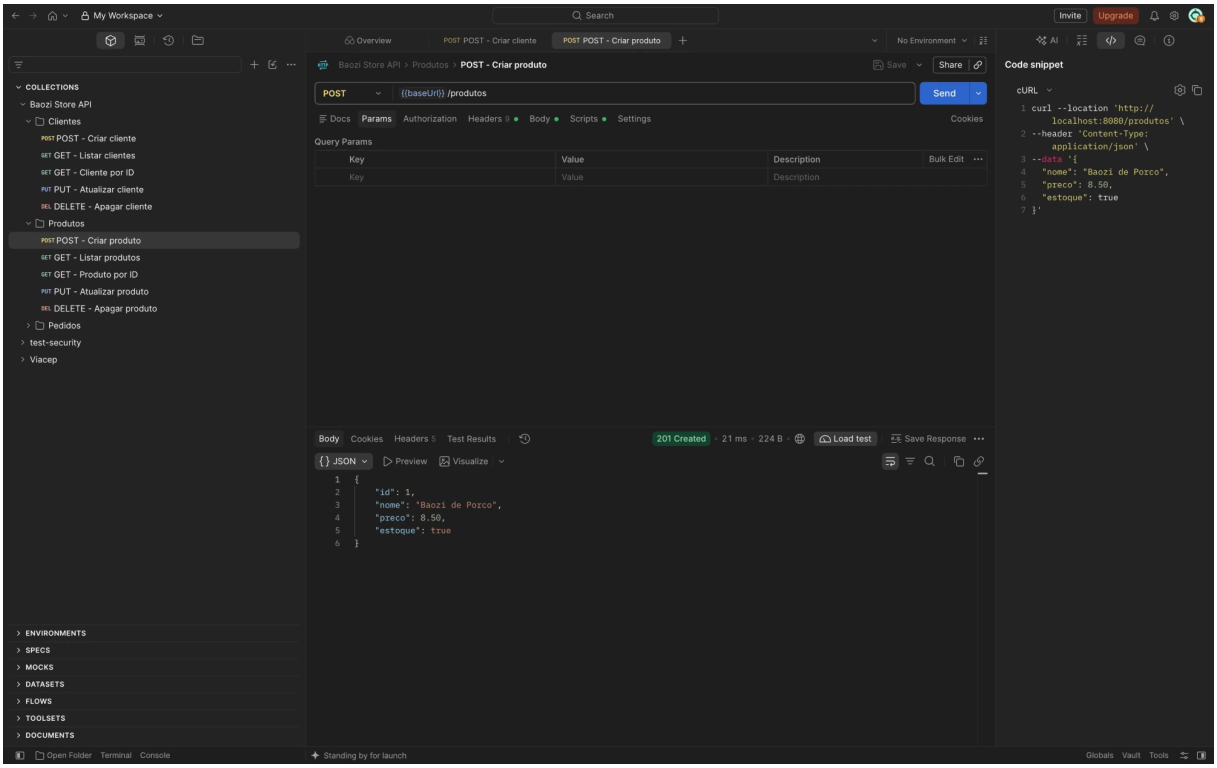
4. Prints do Postman

Testes realizados no Postman com a API rodando em http://localhost:8080.

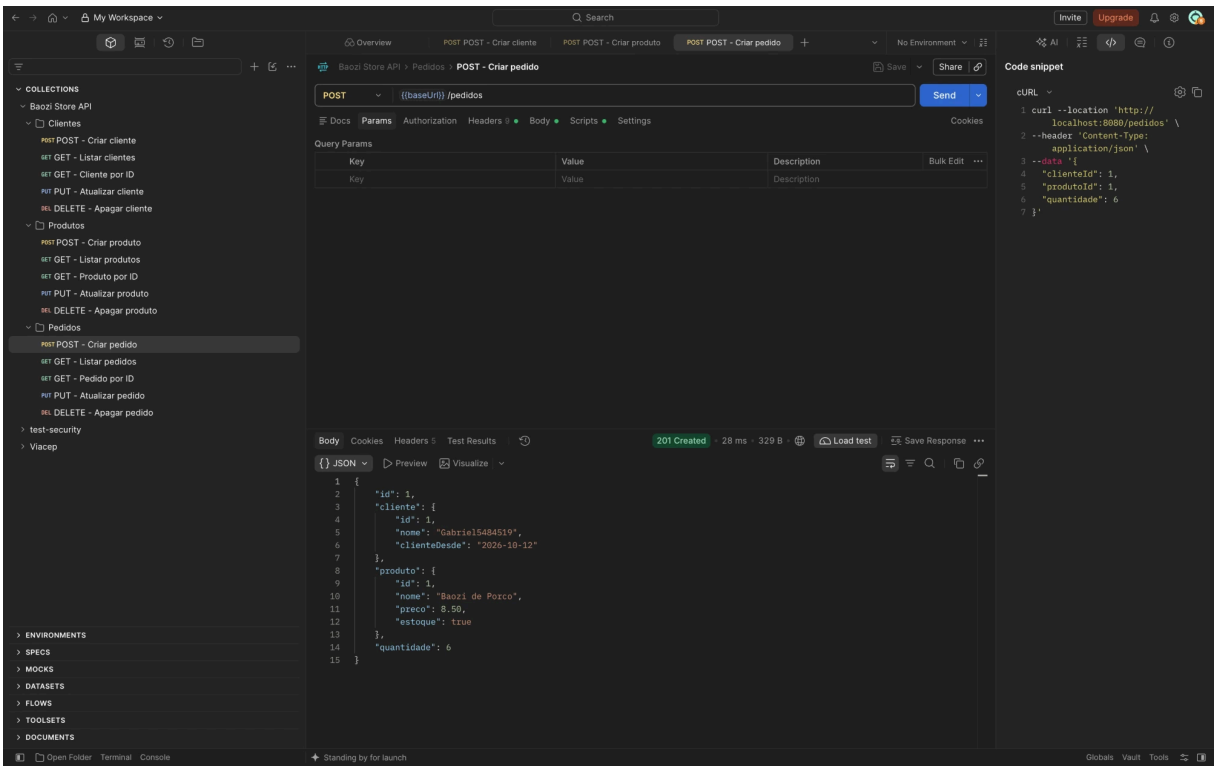
4.1 POST /clientes – criar cliente Gabriel5484519



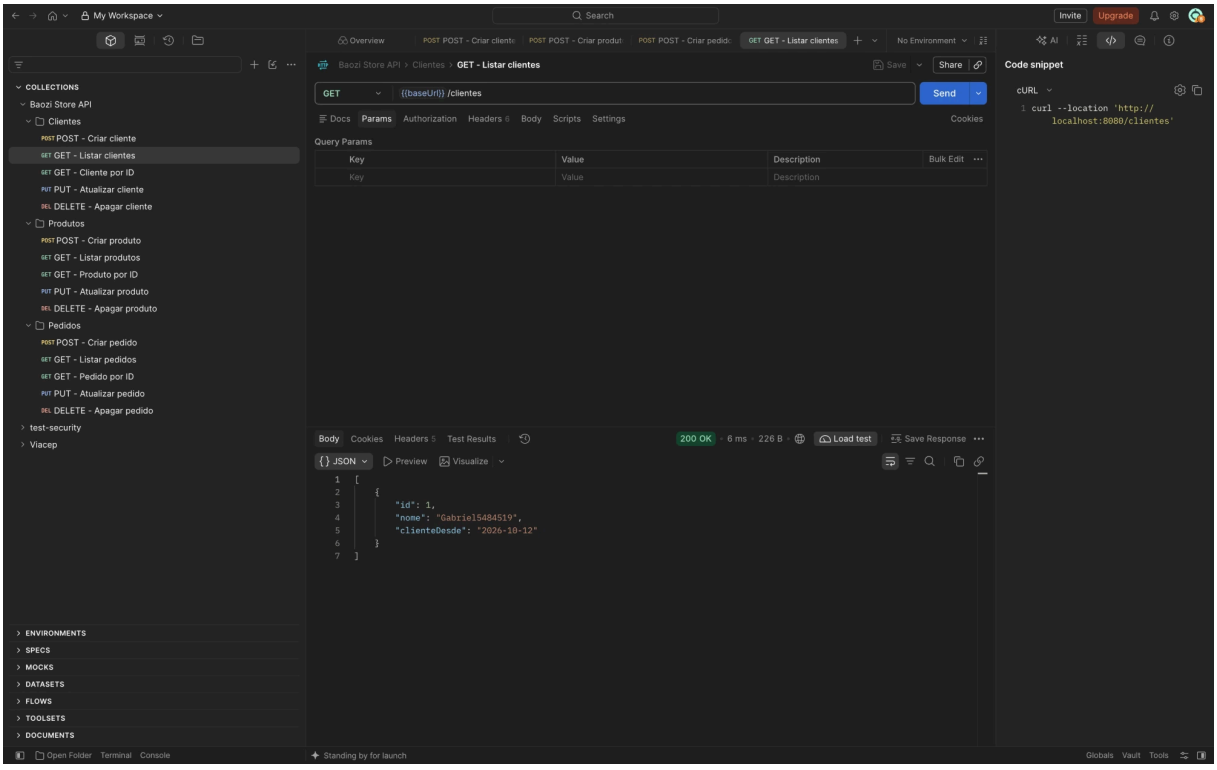
4.2 POST /produtos – criar produto Baozi de Porco



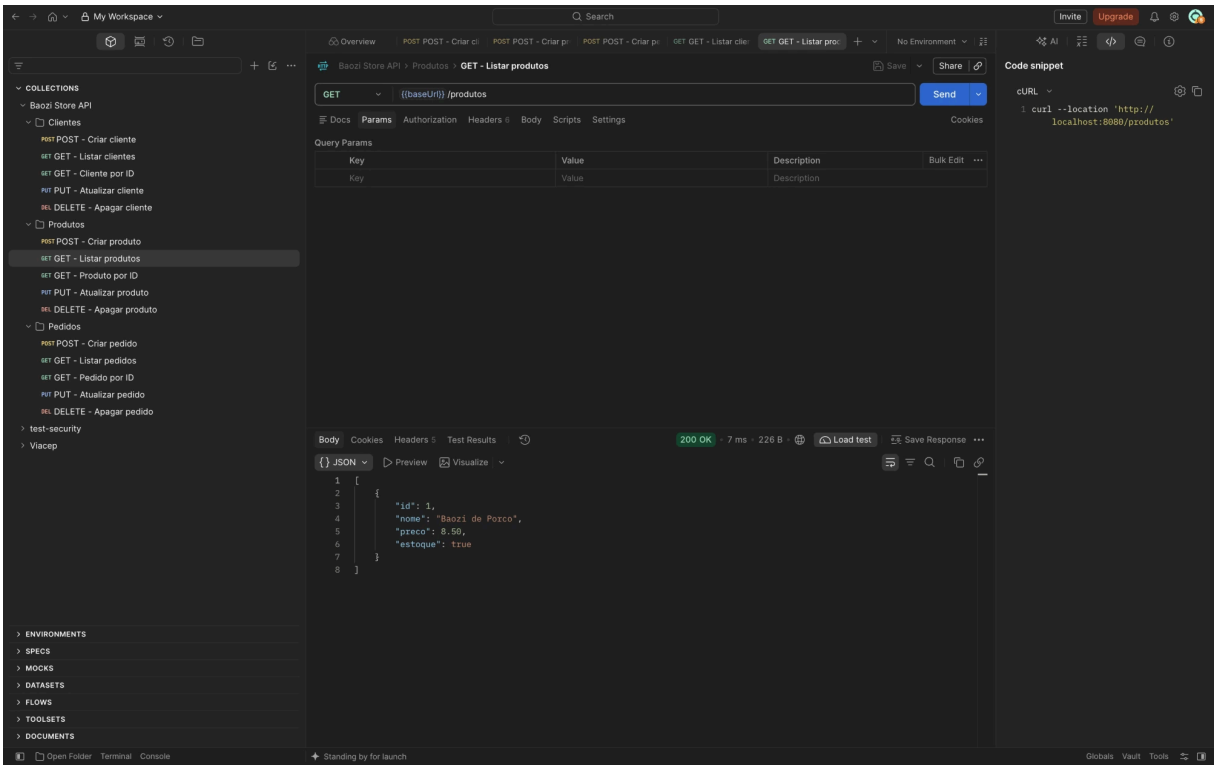
4.3 POST /pedidos – criar pedido de 6 unidades



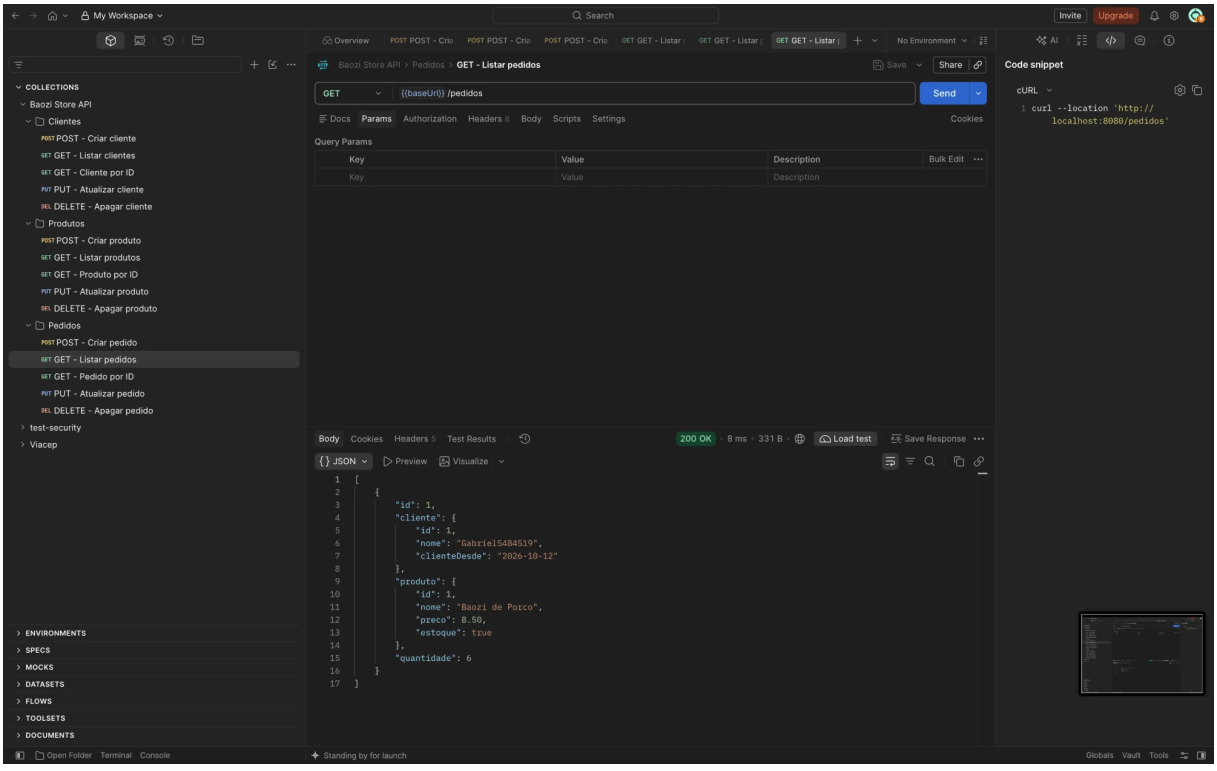
4.4 GET /clientes – listagem geral de clientes



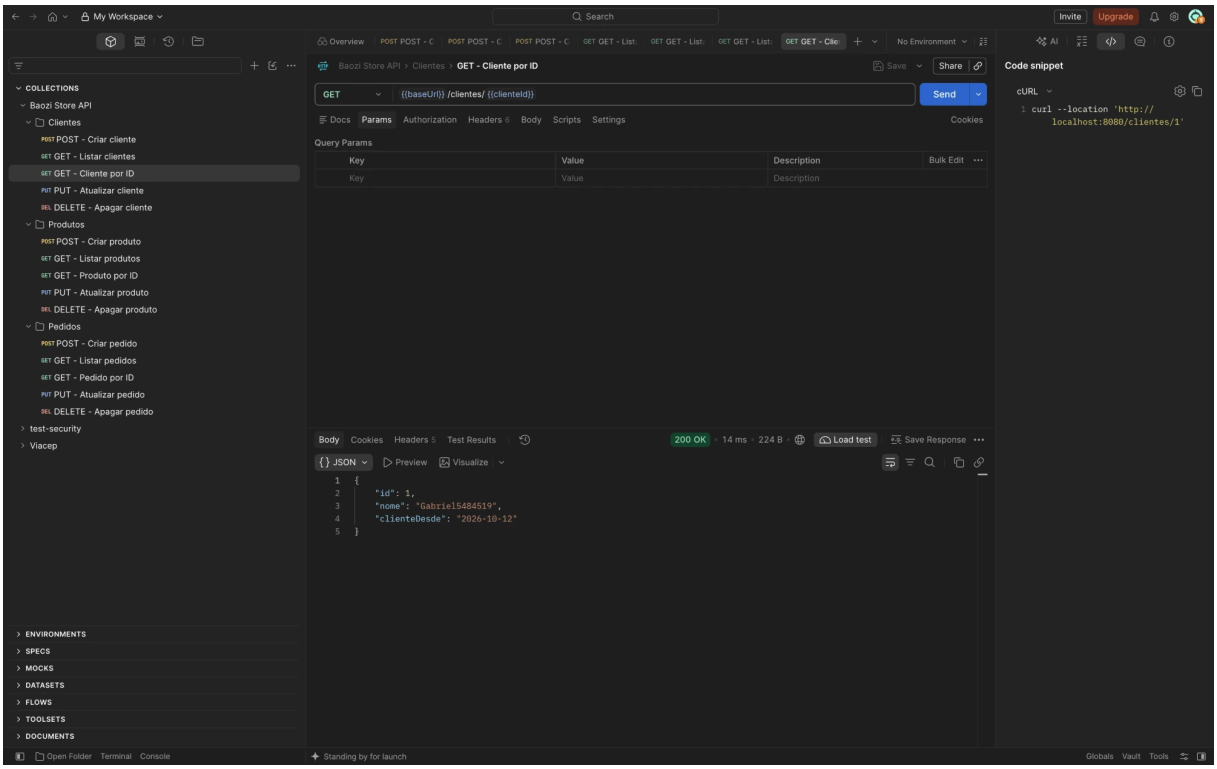
4.5 GET /produtos – listagem geral de produtos



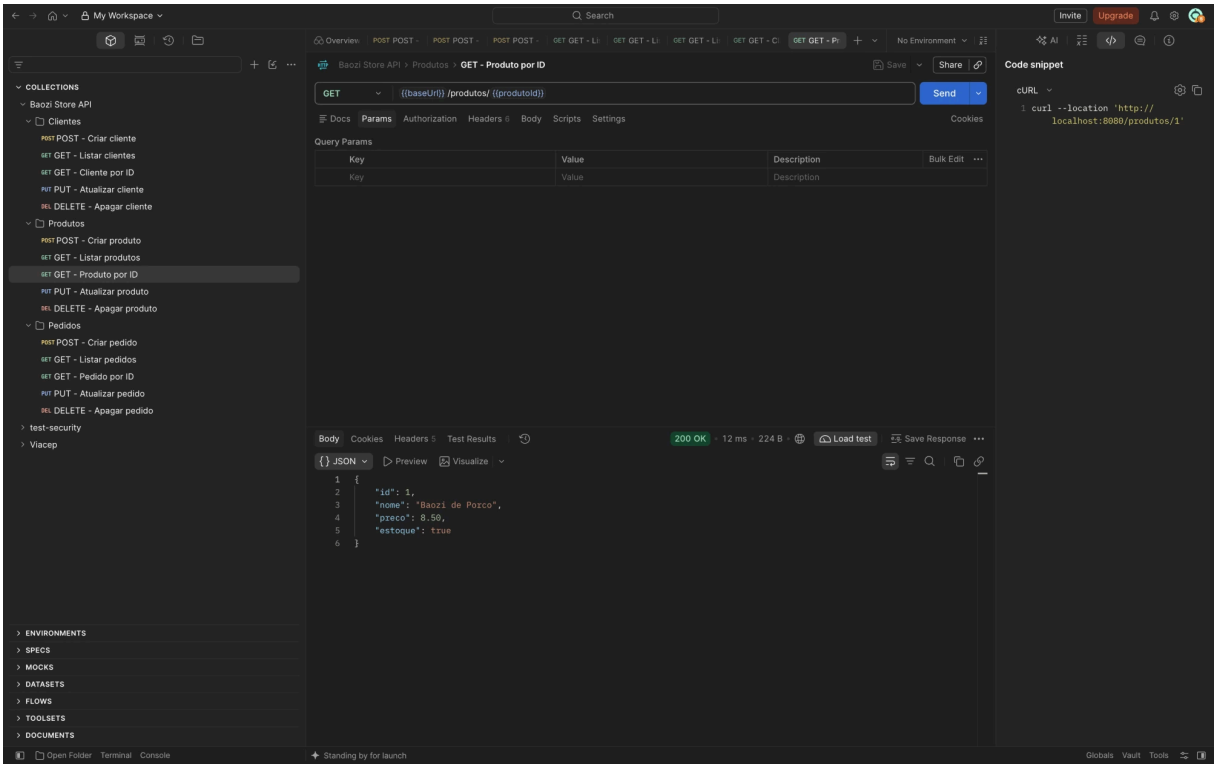
4.6 GET /pedidos – listagem geral de pedidos



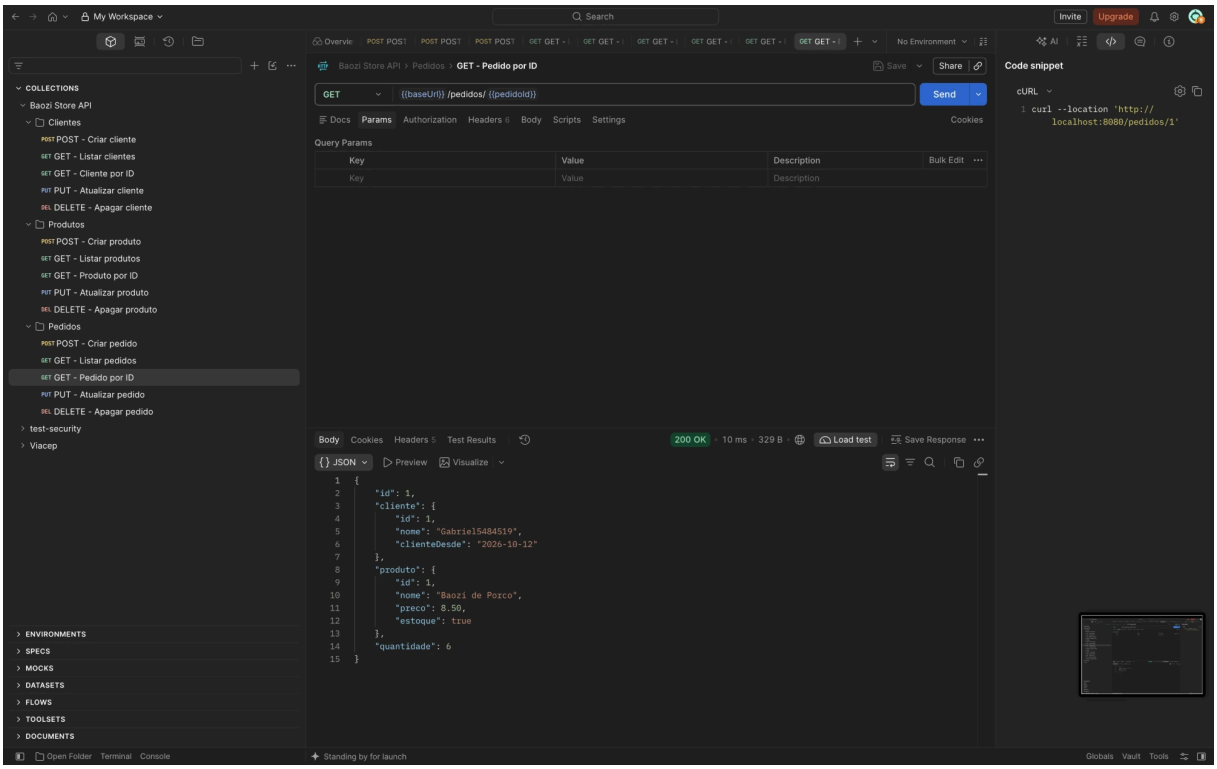
4.7 GET /clientes/{id} – consulta de cliente por ID



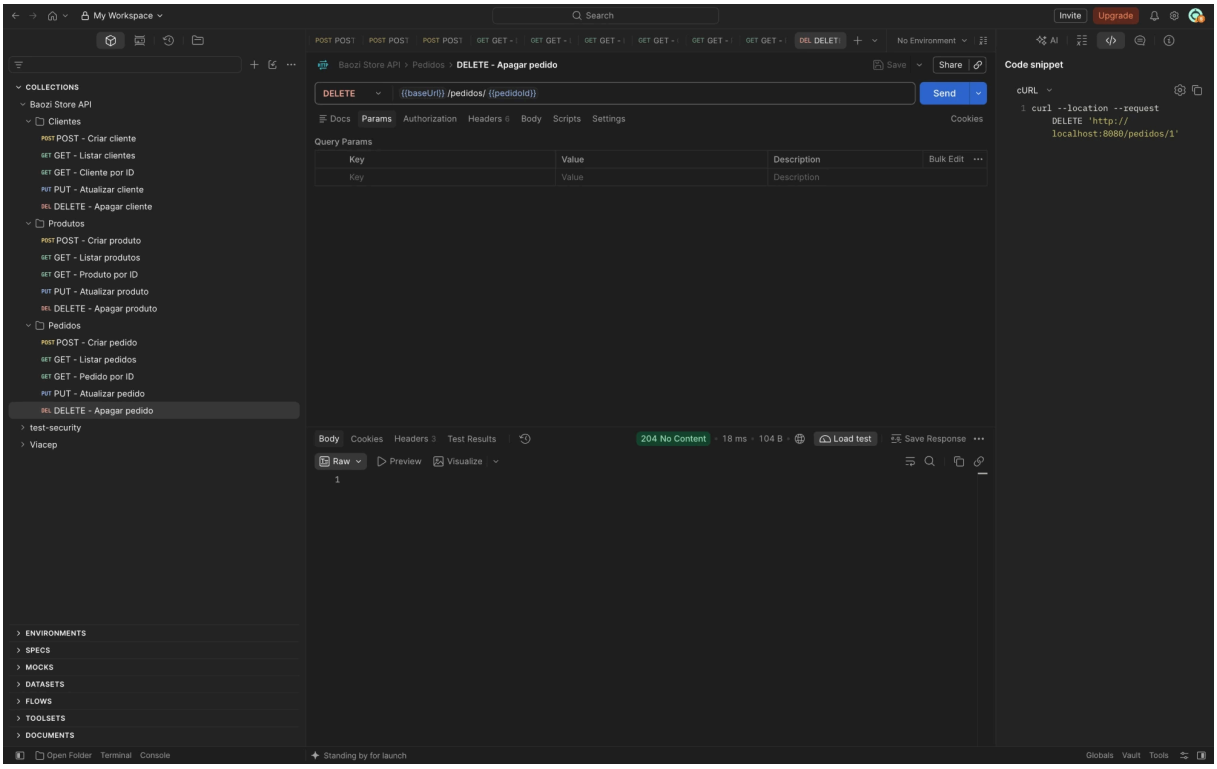
4.8 GET /produtos/{id} – consulta de produto por ID



4.9 GET /pedidos/{id} – consulta de pedido por ID



4.10 DELETE /pedidos/{id} – apagar pedido



5. Link do repositório (GitHub ou similar) contendo o projeto(código fonte)
https://github.com/Gbcavalieri/baozi-store